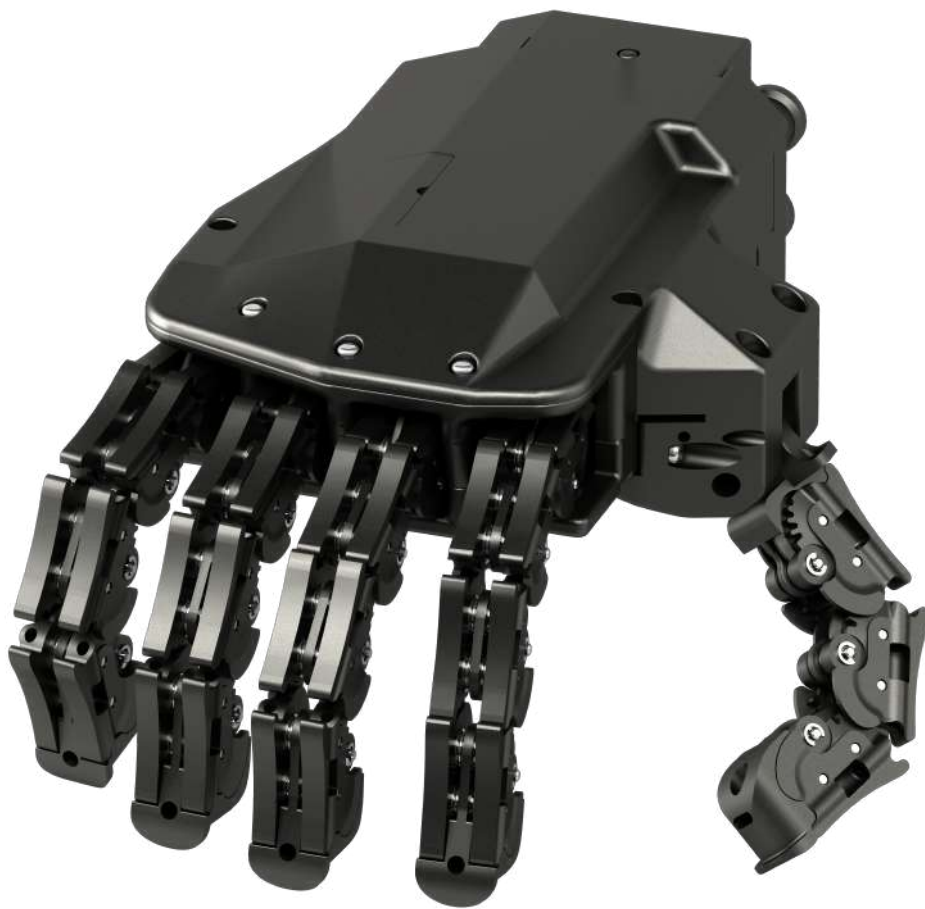




General Manual

PLEASE READ CAREFULLY THESE INSTRUCTIONS BEFORE USE. DO NOT DISCARD: KEEP FOR FUTURE REFERENCE.

Dear customer,

Thank you for purchasing our product.



The present document provides information at best of our knowledge at the time of publication. This document could present differences from the product and it is subject to changes without notice: the latest version is available on our webpage www.qbrobotics.com.

qbrobotics s.r.l. does not assume any responsibility for errors or omissions.

In no case qbrobotics s.r.l. will be responsible for any loss, problems or damages to persons or property arising from the use of this document.

The qb® logo and qbrobotics® are registered trademarks of qbrobotics s.r.l. In the following, the indications of (R) are omitted.

Table of Contents

1 Preface	2
1.1 Using this document	2
1.2 Symbols and conventions	2
1.3 Kit content	2
2 Safety.....	4
2.1 Intended use	4
2.2 Safety instructions	4
2.3 EC Directives on product safety	5
2.4 Environmental conditions	5
2.5 Environmental safety	5
3 Technical data	7
3.1 Mechanical dimensions	7
3.2 Center of Mass	8
3.3 Mechanical characteristics	9
3.4 Electrical characteristics	10
3.5 Tool connector pinout	11
4 Description	13
4.1 General	13
4.2 Characteristics and key features	13
4.3 Standard customization	13
5 Mounting and wiring	14
5.1 Tool mounting	14
5.2 Cable connections	15
6 Software.....	19
6.1 Configuration GUI	19
6.2 ROS	22
6.3 C++ API library	34
7 Maintenance and warranty	39
7.1 General	39
8 FAQ and troubleshooting.....	40
8.1 PC drivers and requirements	40
8.2 Grasping with a custom finger position	43
8.3 Simplified CAD models of the qb SoftHand	44
8.4 The glove is deteriorated/broken	44

8.5	The hand does not reopen correctly	45
8.6	The USB flash drive is not working	46

1 Preface

1.1 Using this document

Please, check that the document release date is before the product purchase date.

The documentation must always be complete and in a perfectly readable state.

Keep the document accessible to the operating and, if necessary, maintenance personnel at all times.

Pass the document to any subsequent owner or user of the product.

1.2 Symbols and conventions

STYLE	DESCRIPTION
 CRITICAL: ...	Identifies information about practice or circumstances that can lead to critical damages on the device and to personal injury. Attentions help you identify a hazard, avoid a hazard, and recognize the consequence.
 WARNING: ...	Identifies information about practice or circumstances that can lead to damages on the device and to personal injury. Attentions help you identify a hazard, avoid a hazard, and recognize the consequence.

STYLE	DESCRIPTION
NOTE: ...	Identifies remarkable information and additional notes.
TIPS: ...	Identifies tips and highlights.
	Identifies the required tool to be used during the described assembly phase.
<code>Monospaced text</code>	Identifies file paths, file names and software functions.

1.3 Kit content

The kit consists of:

- N.1 qb SoftHand Research 24 V;
- N.1 ISO 9409-1-50-4-M6 flange adapter;
- N.1 0.3m direct wrist connection (compatible robots in download area);
- N.1 3m main cable;
- N.1 USB to RS485 converter with extension cord;
- N.1 USB flash drive with user documents and applications;
- N.1 cylindrical pin EN ISO 8734 A d6x14 h6;
- N.4 metrical screws EN ISO 4762 M6x10;
- N.8 metrical screws EN ISO 10642 M3x8;
- N.1 2 mm Allen hex key;
- N.1 5 mm Allen hex key.

Optionals:

- 24 VDC power supply unit;
- ISO 9409-1-40-4-M6 flange adapter;
- ISO 9409-1-31.5-4-M5 flange adapter;
- Kinova Gen3 flange adapter.

2 Safety

2.1 Intended use

The product design is intended for grasping objects in the weight range from 1g to 2 Kg.

Fragile, sharp or sharp-edged objects shall not be grasped.

Objects having the ratio between the biggest and the smallest dimensions $\geq 8\text{mm}$ and the main dimension greater than 300 mm shall not be grasped.

The product, as its name suggests, is intended for research and educational use.

The product is intended for installation on robotic arms: the safety features are established only for use as described in this document.

The safety of the product cannot be guaranteed in case of inappropriate use. One, single, inappropriate use can result in a permanent damage to the safety of the product.

2.2 Safety instructions



WARNING:

- Check that all the content is intact after removing it from the packaging.
- The device can be used only by specially trained staff.
- Disconnect the power supply before installation, cleaning or maintenance operations.
- Make sure that no residual energy remains in the system.
- Always operate the product within the specifications defined.
- Keep away from children and pets. Always set off or unplug when not in use.
- Never use aerosol products, petroleum based lubricants or other flammable products on or near the end-effectors.
- Do not use any damaged power cable, plug, or loose outlet. It may cause damages to the product or injury to people.
- Do not touch electrical components to avoid damages due to electrostatic charges.
- Make sure the end-effector is properly and securely bolted in place and cabled.
- Do not use if damaged or defective. Do not disassemble.
- Do not insert any objects between moving parts of the fingers.
- Keep head and face outside the reach of the end-effector.
- Do not wear loose clothing or jewelry when working with the end-effector.
- Disrespect of these precautions can affect safety of the device.

2.3 EC Directives on product safety

- The following EC directives on product safety must be observed.
- If the product is being used outside the UE, international, national and regional directives must be also observed.

2.3.1 Machinery Directive (2006/42/EC)

Because of their small size, no serious threats to life or physical condition can normally be expected from electric miniature drivers. Therefore, the Machinery Directive does not apply to our products. The products described here are not "incomplete machines", so installation instructions are not normally issued by qbrobotics.

2.3.2 Low Voltage Directive (2014/35/EU)

The Low Voltage Directive applies for all electrical equipment with a nominal voltage of 75 to 1500 V DC and 50 to 1000 V AC. The products described in this device manual do not fall within the scope of this directive, since they are intended for lower voltages.

2.4 Environmental conditions

Wrong environmental and operating conditions can lead to injuries, product damages and/or significant reduction to the product's life.



WARNING:

Any use or application deviating from intended use is deemed to be impermissible misuse. This includes, but is not limited to:

- Use before performing a risk assessment;
- Use outside the permissible operational conditions and specifications;
- Use in not low-dust environment;
- Use in places with high temperature or humidity;
- Use in wet places;
- Use in potentially explosive atmospheres;
- Use in medical and life critical applications;
- Use close to a human's head, face and eye area;
- Use as a climbing aid;
- Use in outdoor applications.

2.5 Environmental safety

The qb SoftHand research must be disposed of in accordance with the applicable national laws, regulations and standards.

All the components of this product have been chosen in accordance with the EU RoHS directive 2011/65/EU: they are produced with restricted use of hazardous substances to protect the environment.

Observe national registration requirements for importers according to EU WEEE Directive 2012/19/EU.



3 Technical data

3.1 Mechanical dimensions

The figure below shows the overall dimensions of the qb SoftHand research, valid for both left and right configurations; distances and tolerances in the drawing are noted in millimeters [mm] and degrees [°].

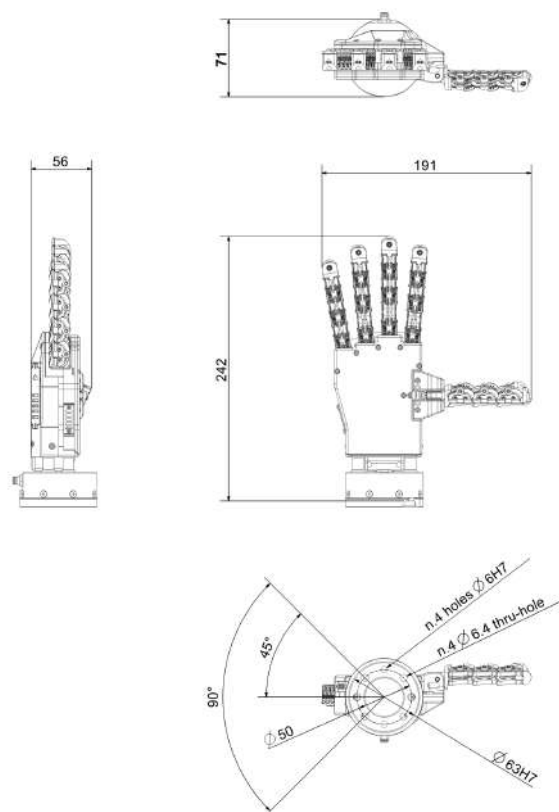


Figure 1. — qb SoftHand drawings



WARNING:

Be aware that the above figure may be out of scale.

3.2 Center of Mass

The following table gives also the position of the Center of Mass of the SoftHand and defines its Coordinate System, like depicted in the following schemes.

	UNIT	RIGHT, OPEN	RIGHT, CLOSED
CoM x coordinate, CoM_X	[mm]	0.5	-3.5
CoM y coordinate, CoM_Y	[mm]	-2.7	-1.3
CoM z coordinate, CoM_Z	[mm]	82.9	76.6
Weight	[kg]	0.770	

Center of Mass, CoM, is evaluated w.r.t. Σ_M considering the hand mounted with its palm facing the ground, cf. the figures below.

For Left SoftHands, CoM_Y has same values but opposite in sign.



NOTE:

All coordinate systems here described follow the orthogonal right-hand rule.

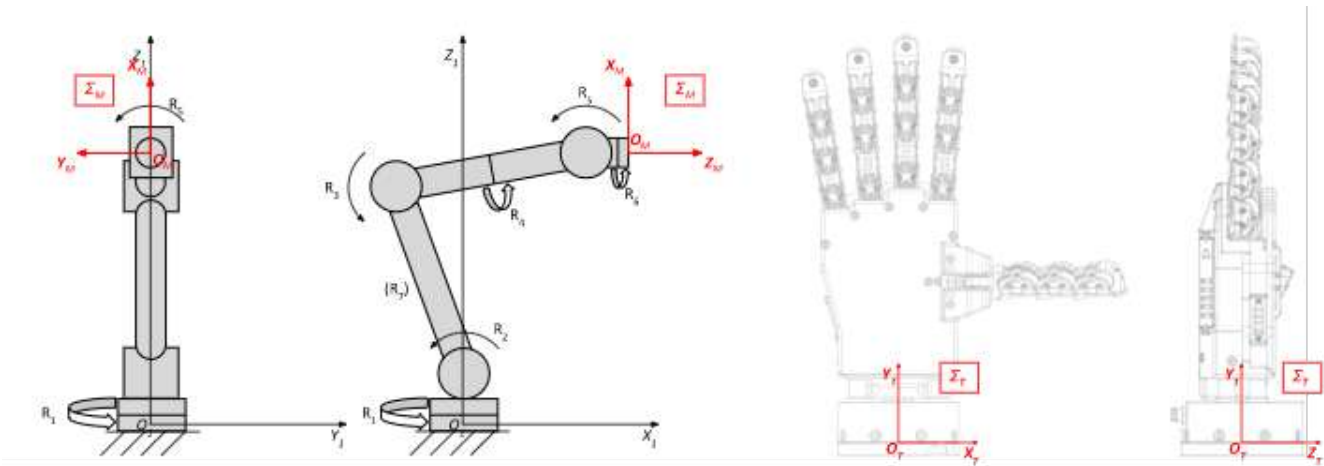


Figure 2. — Robot Tool Coordinate System

On the left, it is represented the mechanical interface coordinate system Σ_M (O_M ; X_M , Y_M , Z_M) of an articulated robot, as defined by ISO 9787:2013. In particular, the center of the coordinate system, O_M , is on the interface surface of the robot tool flange and the Z_M axis is coincident with the tool flange axis of symmetry. R_i is the i -th revolute joint of the robot and (O_1 ; X_1 , Y_1 , Z_1) is the base coordinate system of the robot.

On the right, is represented the Tool Coordinate System, $\Sigma_T (O_T, X_T, Y_T, Z_T)$, of the SoftHand. The center, O_T , coincides with O_M . The Z_T axis is normal to the palm, outgoing positive, and the Y_T axis follows the proximal-distal direction and orientation. This definition implies that Σ_T is the same for left and right SoftHands. So, the positive orientation of X_T axis will be on the side of the thumb in the right hand and on the side opposite to the thumb in the left hand.

3.3 Mechanical characteristics

	UNIT	MINIMUM	MAXIMUM
Pinch grasping payload	[kg]	—	1.10
Power grasping payload	[kg]	—	2.10
Hanging payload	[kg]	—	5
Pinch grasping force	[N]	—	11
Power grasping force	[N]	—	62
Grasping time	[s]	1.1	—
Releasing time	[s]	1.1	—

Look at the graph for details.

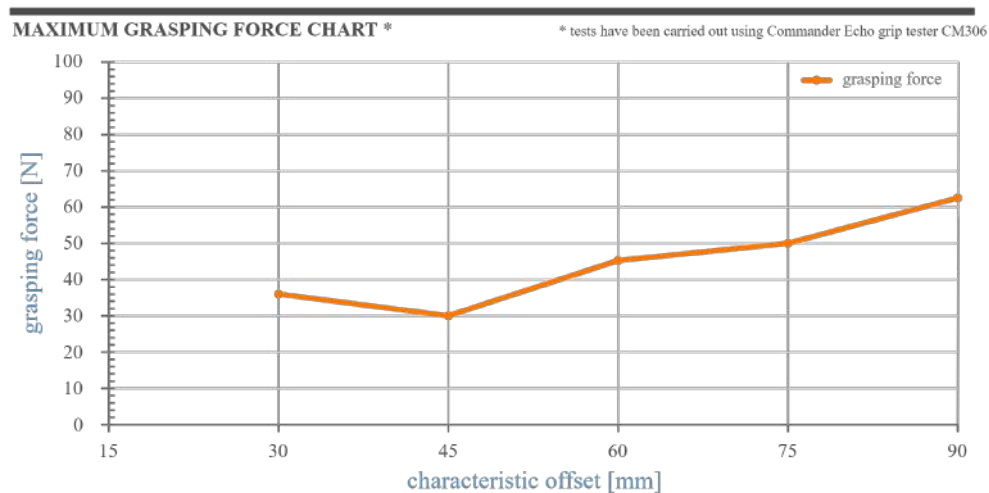


Figure 3. — Maximum grasping force chart

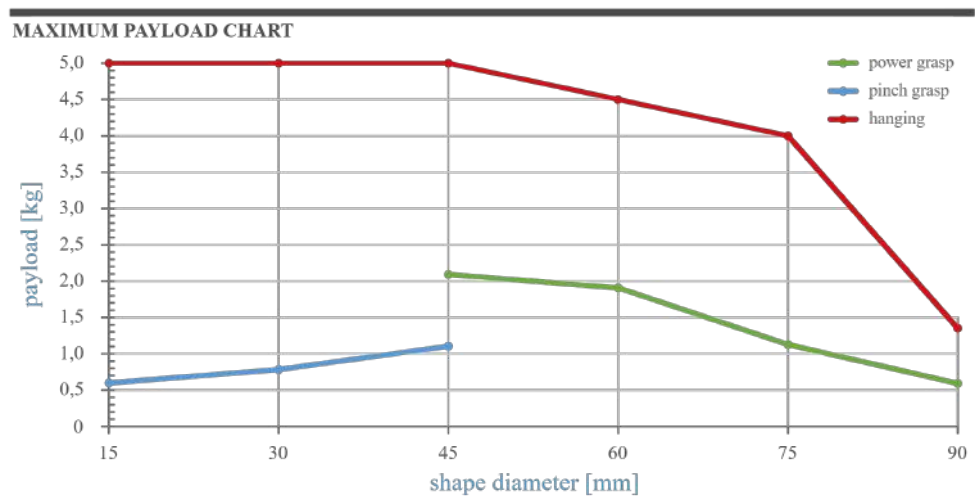


Figure 4. — Maximum payload chart

CHART NOTES	
shapes diameters:	15-30-45-60-75-90 [mm]
cylinder length:	150 [mm]
friction factor:	0,8±0,1 ¹
¹ hand rubber glove vs ABS plastics	

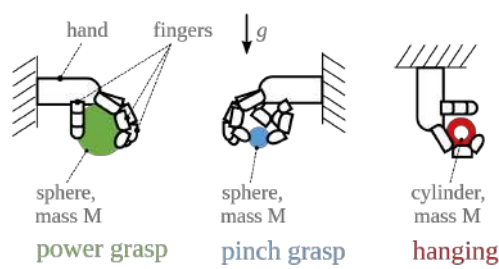


Figure 5. — Chart notes

3.4 Electrical characteristics

		MINIMUM	NOMINAL	MAXIMUM
Power supply voltage	[VDC]	23.1	24	24.8
Power consumption	[W]	1.4	18	30
Current limit	[A]	—	0.75	*
Operational time	[s]	—	—	300
Duty cycle	[%]	—	—	69

		MINIMUM	NOMINAL	MAXIMUM
Operating temperature	[°C]	-15	20	40
Relative humidity **	[%]	5	—	80
Typical noise level	[dB]	—	<50	—

* Starting spike current may be greater, but it is instantaneous.

** Only the non-condensing case is to be considered.

3.5 Tool connector pinout

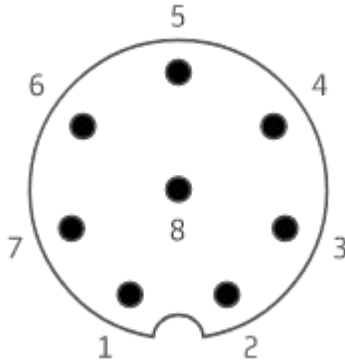
On the qb SoftHand flange there is a 8 pin M8 circular connector, A-coded (Part Number: [1424232](#)¹ or equivalent).

The following cables are compliant and recommended:

- Phoenix contact: 3m free cable end, 8-position, with M8 angled socket — Part Number: [1404192](#)²
- Phoenix contact: 300mm cable, 8-position, M8 angled socket on M8 angled socket — Part Number: [1098315](#)³

The RS485 protocol characteristics are as follow:

- 8 bit;
- no parity;
- 1 stop bit;
- no flow control;
- 2M baud rate.

#	WIRE COLOR	DESCRIPTION	PIN ASSIGNMENT OF M8 CIRCULAR CONNECTOR
1	White	RS485+	
2	Brown	RS485-	
3	Green	—	
4	Yellow	Digital output	
5	Gray	24V ***	
6	Pink	—	
7	Blue	—	

1. <https://www.phoenixcontact.com/online/portal/us?uri=pxc-oc-itemdetail;pid=1424232>

2. <https://www.phoenixcontact.com/online/portal/us?uri=pxc-oc-itemdetail;pid=1404192>

3. <https://www.phoenixcontact.com/online/portal/us?uri=pxc-oc-itemdetail;pid=1098315>

#	WIRE COLOR	DESCRIPTION	PIN ASSIGNMENT OF M8 CIRCULAR CONNECTOR
---	------------	-------------	---

8	Red	GND ***	
---	-----	---------	--

*** 24VDC and GND must be provided by an isolated Power Supply Unit.

**WARNING:**
Any error in the pins connection may cause irremediable damages to the device.

4 Description

4.1 General

qb SoftHand Research is an anthropomorphic robotic hand based on soft-robotics technology. It is flexible, adaptable and able to interact with the surrounding environment, objects, or even humans while limiting the risk of injuring operators, spoiling products, or damaging the robot itself.

Thanks to its soft nature the SoftHand by qbrobotics exploits the principles of synergies in an intrinsically intelligent design that is not only safe w.r.t. unexpected human-robot interaction, but also adaptable to grasp different shaped objects without any change in the control action — i.e. showing an unparalleled level of simplicity and flexibility.

4.2 Characteristics and key features

- Flexibility, adaptivity and robustness thanks to the soft-robotics design;
- 19 anthropomorphic DOFs controlled in one single synergy motion;
- Dislocatable and self-reposition phalanges;
- Up to 60 N grasping force*;
- Up to 2.0 kg maximum payload*;
- Maximum closure time of 1.1 s;
- Total weight of 770 g (including aluminium flange and screws);
- ROS packages and general C++ API available;

* *Grasp forces and payloads highly depend on object dimensions and approaching strategy (cf. [Mechanical Characteristics](#) (see Page 7)).*

4.3 Standard customization

- Mechanical flange adapters for non-standard robot flanges;
- Left and Right chirality configurations.



NOTE:

We may evaluate other customization for special partners; for requests, please contact our [sales team](mailto:sales@qbrobotics.com)⁴.

4. <mailto:sales@qbrobotics.com>

5 Mounting and wiring

5.1 Tool mounting

The qb SoftHand research kit allows you to connect the device to the robot arm.

The device can be mounted on any robot equipped with a mounting interface **ISO 9409-1-50-4-M6**.



Use only the screws provided within the package. Longer screws could damage the robot or the hand!

To assemble the hand on the robot arm, please follow the following instructions:

INSTRUCTIONS

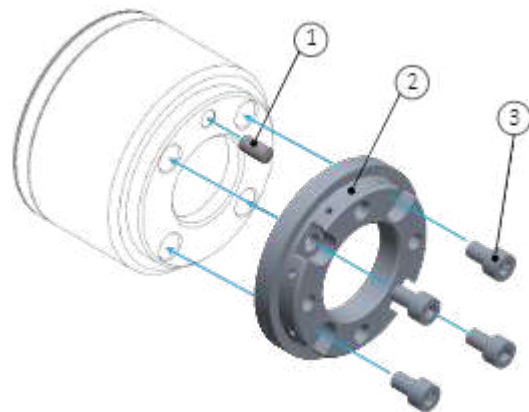
- 1 Insert (1) into the 6 mm hole on the wrist. Center (2) on the wrist diameter 63 mm, taking care that the cylindrical pin (1) fits into one of the four d6 reamed through holes.

Fasten (2) to the robot wrist by tightening the four screws (3).

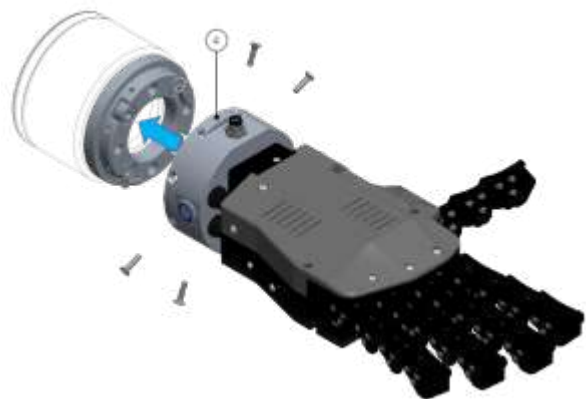
Max Tightening Torque, $TT_{\max}=11.1 \text{ Nm}$.



You need the 5 mm Allen wrench.



- 2 Center (4) on the diameter 63 mm of (2), taking care that the plug fits into one of the four d6 mm through holes.



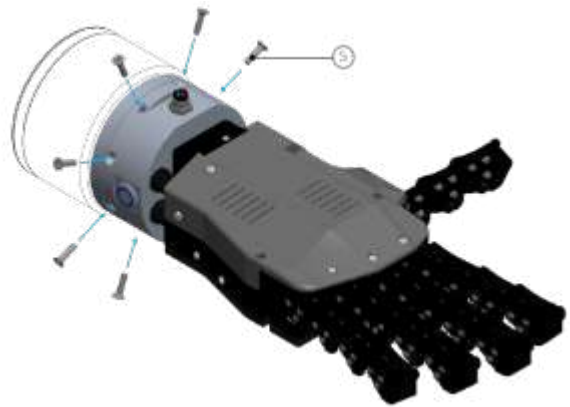
INSTRUCTIONS

- 3 Fix axially the hand by tightening the 8 screws (5).

Max Tightening Torque, $TT_{\max}=2.1 \text{ Nm}$.



You need the 2 mm Allen wrench.



5.2 Cable connections

To power and connect the qb SoftHand to your robots' control box or a workstation computer, please follow the instructions in the table, using the provided kit.



For external power supplies, use only 24VDC with at least 2.5A (60W). The following images include an external power supply (**sold separately**). If your robot's control box allows to power external devices, it is not necessary. In this case, please read the following steps carefully.

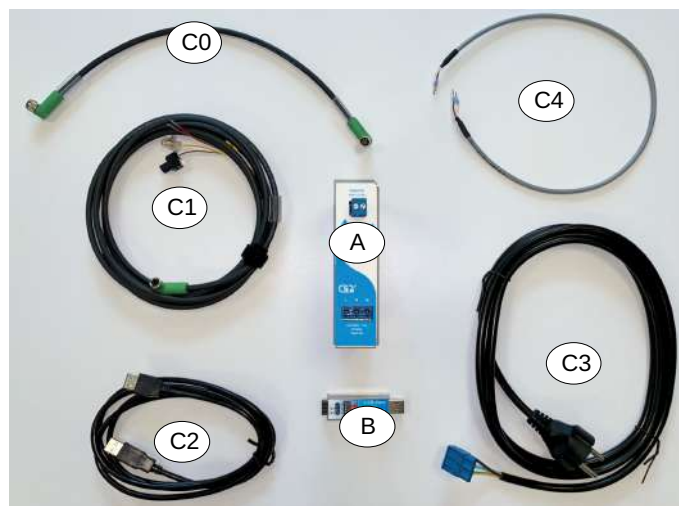


Figure 6. — Connection kit for qbSoftHand Research: optional power supply unit kit (A)+(C3)+(C4), RS485-USB converter (B), main cable (C1), USB-A extension cable (C2).

INSTRUCTIONS

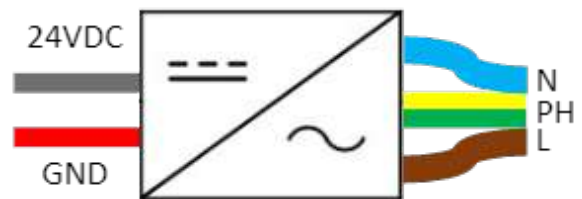
- 1 Connect the main cable (C1) at the RS485-USB converter (B).



- 2 Connect the main cable (C1) to a power supply unit (A) (**Not connected to the power**).

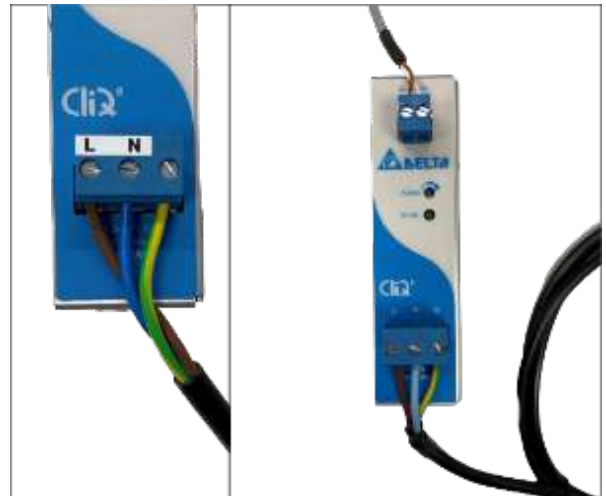


If you have not required the external power supply, the C1 cable has two unconnected red and grey ends. These are used to power the device, so connect the **red wire to the negative (V^-)** and the **grey wire to the positive connector (V^+)**. For example in UR CB series robots you can connect the device to 24 V sockets inside the Digital Inputs panels.



INSTRUCTIONS

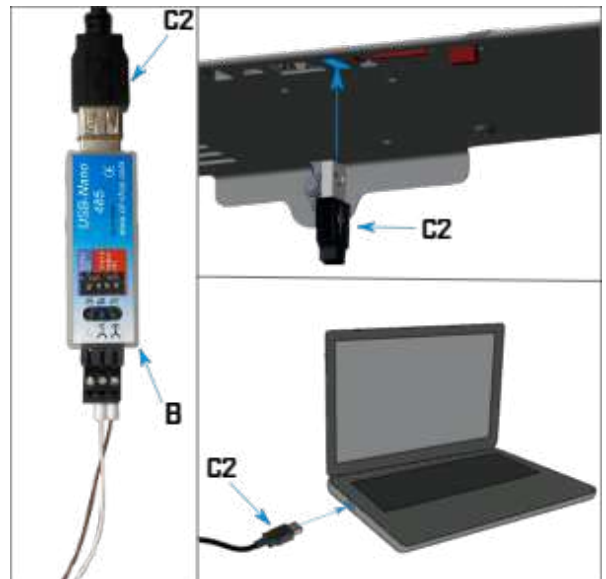
- 3 If you have purchased our OPTIONAL external power supply, connect the power cable (C3) at the power supply unit (A).



- 4 Connect the USB cable (C2) at the RS485-USB converter (B); then connect the other end to a USB type-A port on your PC or robots' control box.



If the qb SoftHand has to be used through C++ APIs or ROS packages, then the USB communication cable has to be connected to the workstation PC.



- 5 Connect the yellow wire of (C1) to a digital input port of the robots' control box.



By pressing the circular button on the flange, the yellow wire is supplied with 24 VDC.
Do not remove the terminal cap unless the yellow wire is connected to the control unit.
Contact [qbrobotics Support Team](mailto:support@qbrobotics.com)⁵ for more informations.

Digital Inputs			
24V	■	24V	■
DI0	■	DI4	■
24V	■	24V	■
DI1	■	DI5	■
24V	■	24V	■
DI2	■	DI6	■
24V	■	24V	■
		DI7	■

5. <mailto:support@qbrobotics.com>

INSTRUCTIONS

- 6 Insert the M8 connector of (C1) into the receptacle on the hand wrist flange. Lock the M8 connector by manually tightening its threaded ring on the body of the receptacle (max tightening torque= 0.2 Nm).



Be careful to insert the connector in the correct direction.
Do not rotate the M8 connector when inserted into the receptacle.
Minimum curve radius of the cable (9) = 51 mm.

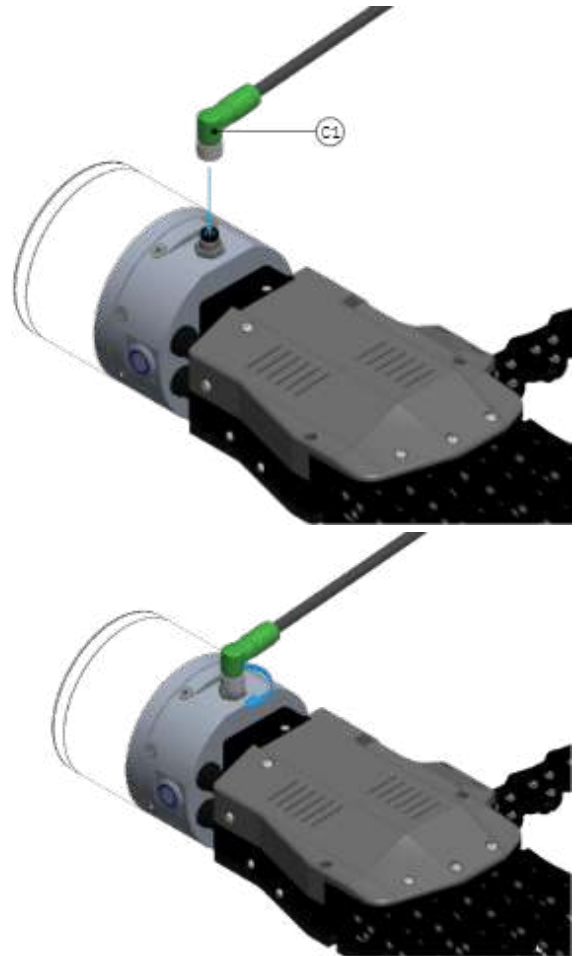


Table 1. — Installation guidelines of the SoftHand research.

6 Software

6.1 Configuration GUI

6.1.1 Installation

You can download our GUI from our [website](#)⁶ (download area).

To install all the requirements for your specific operating system, please refer to the [how-to section](#) (see [Page 40](#)).



For Linux users: use `chmod +x` on the downloaded file and make it executable. Right click on your script and chose **Properties** -> **Permissions** -> **Allow executing file as program**, leaves you with the exact same result as the command in terminal.

6.1.2 Usage

The qb SoftHand GUI (generally referred as *qbtools*) is a simple application to quickly test the basic functionalities of the device through a personal computer. It may be also useful to diagnose — together to our [support team](#)⁷ — troubles about hardware or software components.

The very first thing to do is to click "**Scan Ports**" at the top of the window. If a qb device is properly connected to the system, its serial port name is shown in the drop-down menu on the left.



If there are more than one serial port available, be sure to select the one connected to the qb SoftHand of interest.
If no serial port is found, check out the [installation](#) (see [Page 40](#)) and [mounting](#) (see [Page 14](#)) steps.

When a serial port is found, the "Connect" button becomes available and by clicking on it you scan the serial resource to find every qbrobotics device connected to the system. If there is at least one device found, this operation enables all of the GUI features and it shows a green "**Connected**" label next to the button.

The button **Activate** activates or deactivates the motor driver on the qb SoftHand; the current activation status is shown next to the "**Active**"/"**Inactive**" button;



Remember to power the qb SoftHand before using this GUI.

6. <https://qbrobotics.com/product/qb-softhand-research/>

7. <mailto:support@qbrobotics.com>



Two devices connected to the same master must never have the same ID value. Unexpected behaviors may happen otherwise.

The button of the tab allow:

- **About** : Info about the current GUI.
- **Upgrade** : upgrade the qbSoftHand firmware. Contact the [support team](mailto:support@qbrobotics.com)⁸ for information about the device upgrade.
- **Save Log** : save the device log in a file.
- **Clear Log** : clear the information printed in the panel.
- **Get Info** : shows useful information about the qb SoftHand in the panel. It prints the current state and its settings, e.g. firmware version, parameter values, measurements...
- **Devices ID** : shows a list of device IDs connected to the system; by selecting one of them from the drop-down menu you are able to query that specific device — i.e. requesting information from and sending commands to it;
- **Selected Device ID** : shows the device ID that is currently connected and usable;
- **New ID** : sets a new ID value from the related drop-down menu to the connected qb device; "Current ID" is updated accordingly;

The other tab is described in the following section.

8. <mailto:support@qbrobotics.com>

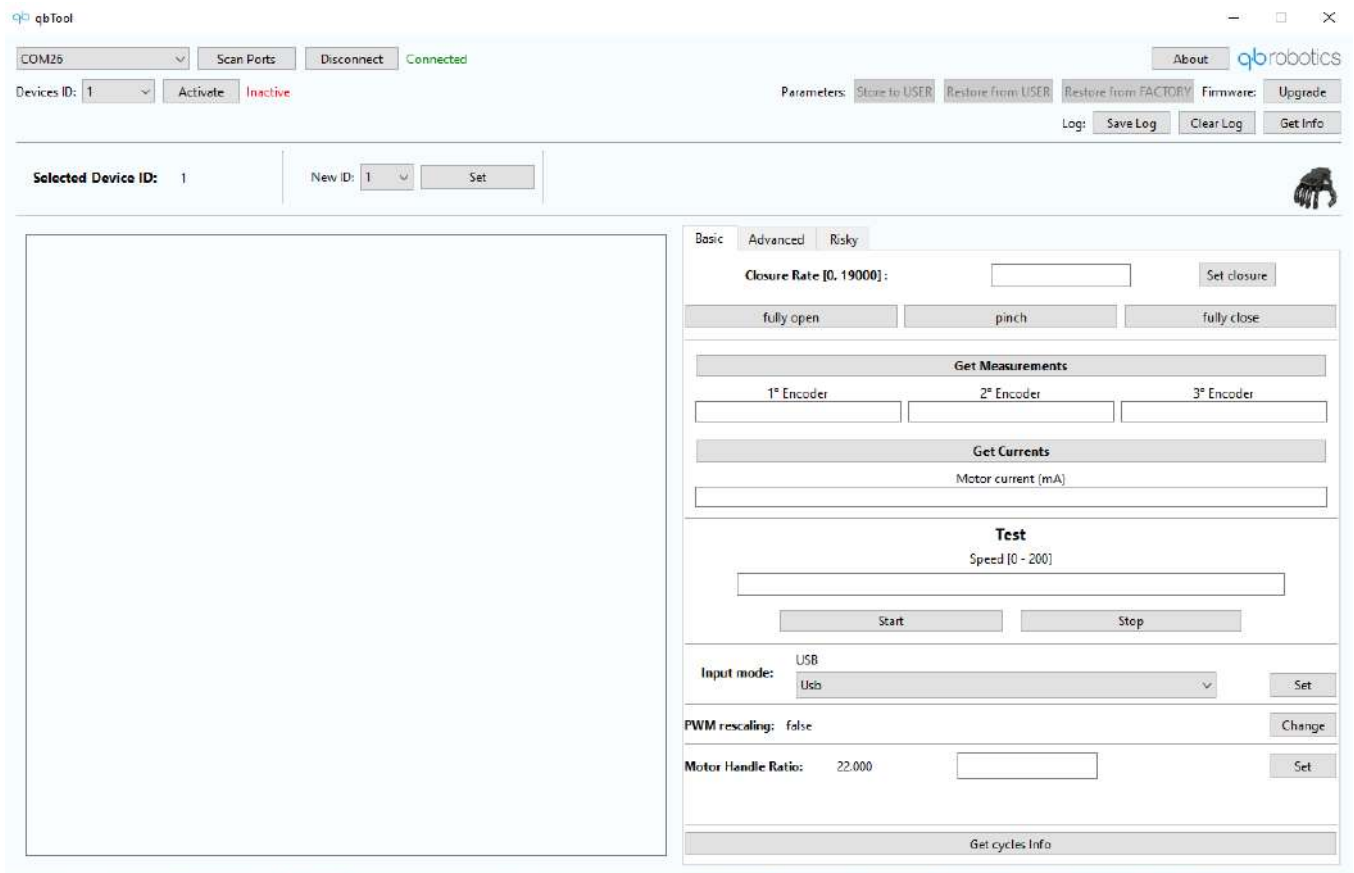


Figure 7. — Overall view of the qbtools GUI

6.1.3 Basic tab



The motors must be activated first to be able to send references to the them.

Measurements and Currents: the two buttons “Get Measurements” and “Get Currents” enable an auto-refreshing thread that shows respectively the encoder measurements or the motor current (in milliamperes) in the fields below;

Input and sliders: It is possible to set the position command to motors.

6.2 ROS



WARNING:

This `README` is basically a mirror of the [qb SoftHand ROS wiki](https://wiki.ros.org/Robots/qbhand)⁹ and it is supplied only for offline documentation. Please, refer to the online wiki whenever you can; it is our main and most updated reference for *qb SoftHand* ROS related application.

6.2.1 Installation

Requirements

If you have never set it up, you probably need to add your linux user to the `dialout` group to grant right access to the serial port resources. To do so, just open a terminal and execute the following command:

```
sudo gpasswd -a <user_name> dialout
```

where you need to replace the `<user_name>` with your current linux username.



NOTE:

Note: don't forget to logout or reboot.

Ubuntu Packages

If you prefer to leave your catkin workspace as it is, you can simply install all the ROS packages from the Ubuntu official repositories:

```
sudo apt update
sudo apt install ros-<ros_distro>-qb-hand
```



WARNING:

ROS **kinetic** distribution is now EOL(end-of-life) so the `qb_hand` packages installed with the previous commands are not updated.

9. <https://wiki.ros.org/Robots/qbhand>

**WARNING:**

This ROS package does not work without **qb_device** one. Please ensure to run:

```
sudo apt install ros-<ros_distro>-qb-device
```

before installing qbSoftHand ros packages.



These packages may not always be updated, as they are only synchronised at certain intervals. To keep up-to-date with our public releases, install the packages with the source code, as described in the section following section (*Sources*).

Sources

**WARNING:**

Since you are interested in the ROS interfaces for our devices, it is assumed that you are familiar at least with the very basics of the ROS environment. If not, it might be useful to spend some of your time with [ROS](#)¹⁰ and [catkin](#)¹¹ tutorials. After that, don't forget to come back here and start having fun with our Nodes.

Install the *qb SoftHand* packages for a ROS user is straightforward. Nonetheless the following are the detailed steps which should be easy to understand even for ROS beginners:

1. Clone both the `qb_device` and `qb_hand` packages to your Catkin Workspace, e.g. `~/catkin_ws` :

```
cd ~/catkin_ws/src
git clone --recurse-submodules https://bitbucket.org/qbrobotics/qbdevice-ros.git
git clone https://bitbucket.org/qbrobotics/qbhand-ros.git
```

2. Compile the packages using `catkin` :

```
cd ~/catkin_ws
catkin_make
```

If you were not familiar with ROS you should be happy now: everything is done! Nonetheless, if you encounter some troubles during the compilation, feel free to ask for support on [our Bitbucket](#)¹².

10. <https://wiki.ros.org/ROS/Tutorials>

11. <https://wiki.ros.org/catkin/Tutorials>

12. <https://bitbucket.org/qbrobotics/qbhand-ros/src>

**NOTE:**

Depending on your ROS installation, you may need some extra packages to properly compile the code. Please, be sure that you have already installed at least `ros-<ros_distro>-ros-controllers`, `ros-<ros_distro>-transmission-interface`, `ros-<ros_distro>-joint-limits-interface`, `ros-<ros_distro>-combined-robot-hw`, and their dependencies (e.g. use `sudo apt install <ros-pkg>`).

Device Setup

Connect a *qb SoftHand* to your system is basically a matter of plugging in a USB cable. Nonetheless, **read carefully** the manual to understand all the requirements and advice about either single-device or chained configurations.

6.2.2 Usage

As shown in the following picture there are two distinct configurations to control several *qb SoftHand* devices connected to the system:

- The first (and recommended) groups all the Hardware Interfaces together (thanks to the [combined_robot_hw](https://wiki.ros.org/combined_robot_hw)¹³) and exploits them as a unique robot system. We have called it "*synchronous*" just to point out that every sequence of reads and writes is always done in the same predefined order.
- The second mode treats every device as an independent Hardware Interface with its dedicated ROS Node which executes the control loop independently w.r.t. the rest of the system, i.e. "*asynchronously*".

Mixed configurations can be also achieved through a proper setup. In such a case we can think of synchronous sub-systems which execute asynchronously w.r.t. each other.

**NOTE:**

In a single-device system the synchronous mode is a nonsense.

13. https://wiki.ros.org/combined_robot_hw

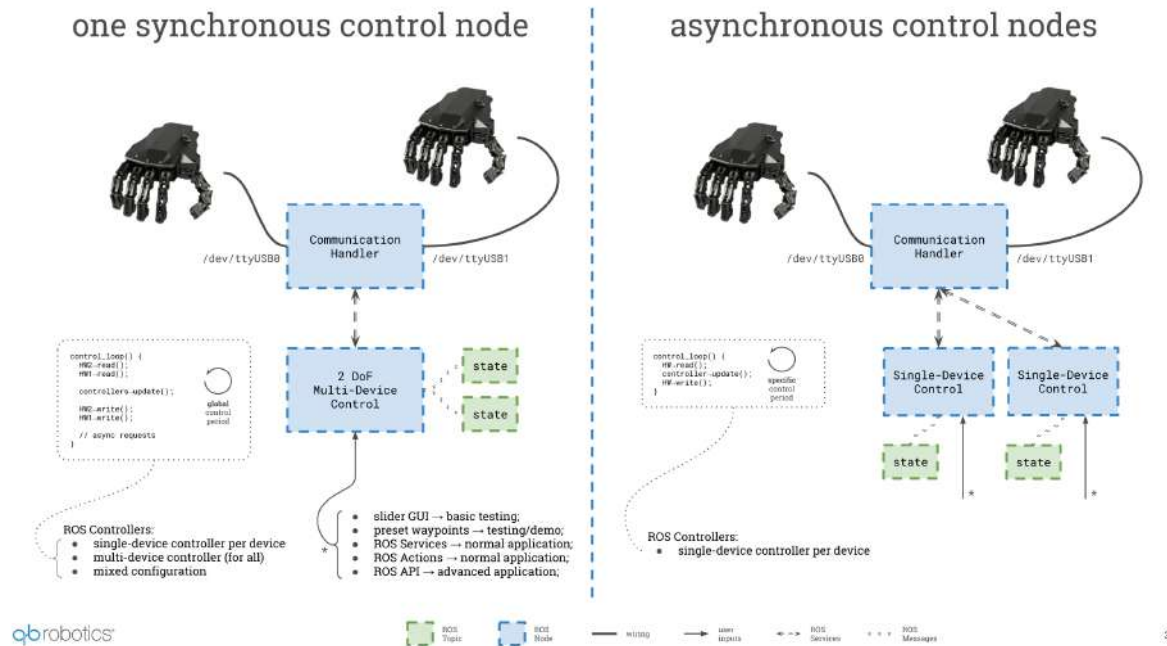


Figure 8. — qb SoftHand ROS control mode schemes

In both cases there is always one central Node which manages the shared resources for the serial communication (e.g. one or many USB ports) and which provides several ROS services to whom wants to interact with the connected devices. This Node is called *Communication Handler* and it is usually started in a separate terminal.



CRITICAL:

Please remember that in a multi-device configuration, each *qbrobotics* device connected to your system **must have a unique ID**. In order to properly set it, you have to use our [GUI](#) (see [Page 19](#)).

Details

To understand what is hiding under the hood, have a look at the C++ classes overview which sums up all the main concepts of our ROS packages:

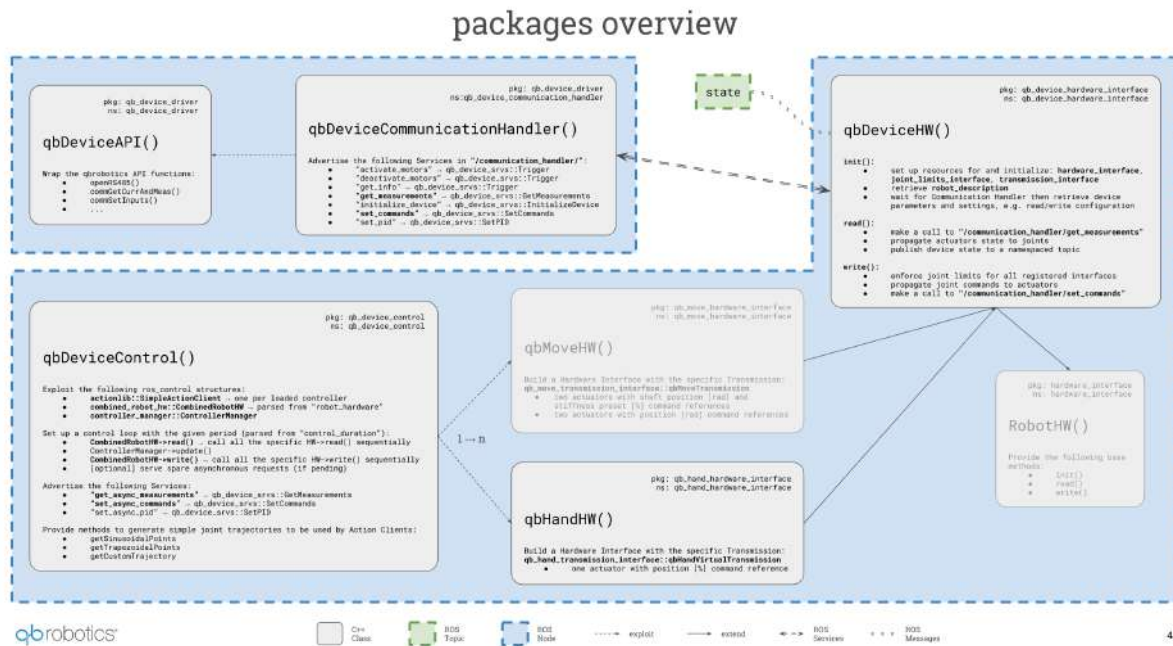


Figure 9. — qb SoftHand ROS packages and classes overview

Communication Handler

The Communication Handler Node has no parameters to be set, therefore it is always launched like this:

```
roslaunch qb_device_driver communication_handler.launch
```

On start, it scans the serial communication resources connected to your system and shows a list of the devices it has found. By default, it never scans again for new devices, apart from asking it explicitly during the initialization of a control Node.

This is a simple example when starting the Communication Handler with two *qbrobotics* devices connected on two distinct USB ports:

```
[ INFO] [1524044523.511369300]: [CommunicationHandler] handles [/dev/ttyUSB1].
[ INFO] [1524044524.426984697]: [CommunicationHandler] handles [/dev/ttyUSB0].
[ INFO] [1524044525.218613760]: [CommunicationHandler] has found [2] devices connected:
[ INFO] [1524044525.218696997]: - device [1] connected through
[/dev/ttyUSB0]
[ INFO] [1524044525.218736612]: - device [2] connected through
[/dev/ttyUSB1]
```

When the Communication Handler is on, it provides all the Services required to interact with the connected devices: e.g. *get info or measurements*, *activate or deactivate motors*, *set commands*, and even more... A detailed description of the services can be found in the [qb_device_driver](#)¹⁴ package wiki.

16. https://wiki.ros.org/qb_chain_control

All the control modes are initialized in the same manner but with distinct command line arguments. The default single-device control Node which brings everything up and simply waits for commands on the above-mentioned Action topic is the following:

```
roslaunch qb_hand_control control.launch standalone:=true
activate_on_initialization:=true device_id:=<actual_device_id>
```

The arguments explained

- `activate_on_initialization [false]` : Activates the motor at startup (the device will not move since the first command reference is received).
- `device_id [1]` : Each device has its own ID, you need to set the one of the actual device connect to your system.
- `standalone [false]` : Starts the Communication Handler together with the control Node. If you set this to `false` (or remove it since the default value is `false`), you need to launch the Communication Handler in a separate terminal.

It is worth noting that the activation of the motor can be postponed to improved safety if you are not aware of the state of the system at startup. To do so just set `activate_on_initialization:=false` (or remove it since the default value is `false`) and make a call to the Communication Handler `activate_motors` Service, when your system is ready, e.g. as follows:

```
rosservice call /communication_handler/activate_motors {"id: <actual_device_id>,
max_repeats: 0"}
```

Additional arguments

- `control_duration [0.01]` : The duration of the control loop expressed in seconds.
- `get_currents [true]` : Choose whether or not to retrieve current measurements from the device.
- `get_positions [true]` : Choose whether or not to retrieve position measurements from the device.
- `get_distinct_packages [true]` : Choose whether or not to retrieve current and position measurements from the device in two distinct packages.
- `max_repeats [3]` : The maximum number of consecutive repetitions to mark retrieved data as corrupted.
- `set_commands [true]` : Choose whether or not to send command positions to the device.
- `set_commands_async [true]` : Choose whether or not to send commands without waiting for ack.
- `use_rviz [false]` : Choose whether or not to use rviz. If enabled you should see a virtual hand on screen performing a similar behavior.

- `use_without_robot [true]` : Choose whether or not to bring up robot description (e.g. when using with other robots is better to set it apart).

**NOTE:**

Be aware that the *qb SoftHand* is desensorized and therefore it is not possible to know exactly the position of each finger: the screen visualization is just the result of an estimation of the closure value and may differ from the real configuration of your *qb SoftHand* (e.g. when grasping an object).

After launching the node, in order to test the qb SoftHand functioning, open a new terminal window and paste the following lines:

```
rostopic pub -1 /qbhand1/control/qbhand1_synergy_trajectory_controller/command
trajectory_msgs/JointTrajectory "header:
  seq: 0
  stamp:
    secs: 0
    nsecs: 0
  frame_id: ''
  joint_names: ['qbhand1_synergy_joint']
  points:
    - positions: [1]
      velocities: [0]
      accelerations: [0]
      effort: [0]
      time_from_start: {secs: 1, nsecs: 0}"
```

Now the qb SoftHand should close. You can use the previous message structure to send control messages to qb SoftHand. The following parameters are important to properly control the device:

- `positions` - accepts values in range [0,1] - e.g. 0 open the qb SoftHand, 1 close it, 0.5 half close, ...
- `time_from_start` - indicates the time for concluding the actions. It sets indirectly the velocity of the qb SoftHand motion.

The other parameters can remain as they are.

**NOTE:**

In the previous example it is assumed that the qbSoftHand has ID set to **1**.

The followings are particular control modes which are enabled with few parameters, but the concepts of this paragraph hold for all of them.

1. GUI Control

This control mode is the one suggested to test that everything is working as expected. You are able to open and close the *qb SoftHand* interactively, but nothing more than this.



NOTE:

You will probably need this only the very first times and for debugging.

To start this mode just add `use_controller_gui:=true` to the general `roslaunch` command (be sure that the opposite `use_waypoints` is not used).

After a while a GUI should appear to screen with two empty drop-down menus, a red enable button below them, and a *speed scaling* slider at the bottom.

1. Select the *Controller Manager* namespace from the left menu, e.g. `/<robot_namespace>/control/controller_manager` (where `<robot_namespace>` is an additional argument of the launch file needed with several devices). This enables the right menu which provides all the controllers available for the connected device.
2. Select the *qb SoftHand* controller from the second drop-down menu and enable it through the circular button.
3. A single slider will appear in the GUI to control the closure of the hand, which ranges from `0` (hand completely open) to `1` (hand completely closed). Move the slider to perform a complete grasp or to partially open/close the *qb SoftHand*. You can also vary the speed through the bottom *speed scaling* slider if you like a faster/slower motion. No other timing constraints can be set in this mode.

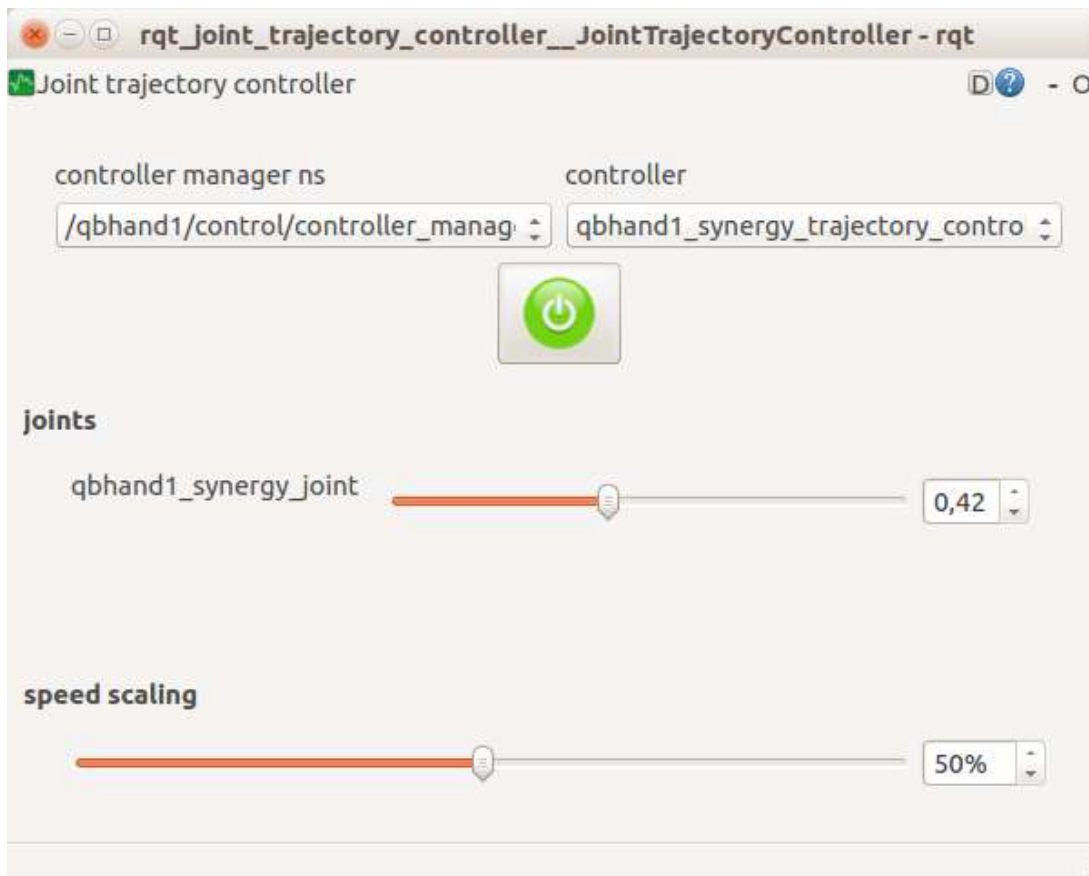


Figure 11. — *qb SoftHand ROS GUI control*

2. Waypoint Control

This control mode is a bit more structured and useful than the previous: it allows to set a fixed trajectory of any number of position waypoints (with timing constraints) and set the robot to cycle infinitely on it (because of the loop it is recommended to set the first and last waypoint in a similar configuration to avoid unwanted sudden changes).

To start this mode just add `use_waypoints:=true` to the general `roslaunch` command (be sure that the opposite `use_controller_gui` is not used). You won't see any control interface in this case but the *qb SoftHand* should start moving according to the given trajectory, parsed from a yaml file located at

```
<robot_package>_control/config/<robot_name>_waypoints.yaml
```

where `robot_name` and `robot_package` are two additional launch file arguments.

Customization

You can modify the waypoint trajectory to replicate the behavior you want: either change the `<robot_package>_control/config/<robot_name>_waypoints.yaml` or add another custom

application-specific file in the `config` directory. In the second case you need to set the argument `robot_name` properly when launching the command from the terminal.

The waypoint configuration is as follows:

```
# Waypoints describe the desired motion trajectory:
# - time [s]: can be either a single value or an interval for which joint_positions
  hold
# - joint_positions:
#   - hand closure [0,1];
# - joint_velocities: optional, use it only if you want a nonzero value
#   - hand closure [0,1]/s;
# - joint_accelerations: optional, use it only if you want a nonzero value
#   - hand closure [0,1]/s^2;
#
# It is worth noting that, if specified, joint_positions, joint_velocities and
  joint_accelerations must be of size one.

waypoints:
-
  time: [1.0]
  joint_positions:
    <device_name>: [0.0]
-
  time: [2.25, 2.75]
  joint_positions:
    <device_name>: [0.8]
-
  time: [4.0]
  joint_positions:
    <device_name>: [0.5]
  joint_velocities:
    <device_name>: [-0.5]
-
  ...
```

3. API Control

If you need a complex (i.e. real) control application, e.g. the *qb SoftHand* is mounted on a robot which uses computer vision aid to grasp objects, the previous two control modes don't really help much. What we provide for real applications is the full ROS libraries to manage and control the *qb SoftHand*.

You have to dig into the `qb_hand`¹⁷ package documentation and find what better suits for your needs, e.g. extend the `qbDeviceControl` class provided, or even redesign some of its parts by following an approach similar to ours.

17. https://wiki.ros.org/qb_hand

**NOTE:**

Our recommendation is to use as much as possible our resources, classes and macros to help you while developing your application. Don't reinvent the wheel!

At last, if you come up with something useful for the whole community, it will be amazing if you propose your improvement with a Pull Request in the package of interest on [our Bitbucket](#)¹⁸.

6.2.3 Support, Bugs and Contribution

Since we are not only focused on this project it might happen that you encounter some trouble once in a while. Maybe we have just forget to think about your specific use case or we have not seen a terrible bug inside our code. In such a case, we are really sorry for the inconvenience and we will provide any support you need.

To help you in the best way we can, we are asking you to do the most suitable of the following steps:

1. It is the first time you are holding a *qb SoftHand*, or the first time you are using ROS, or even both: it is always a pleasure for us to solve your problems, but please consider first to read again the instructions above and the ROS tutorials. If you have ROS related questions the right place to ask is [ROS Answers](#)¹⁹.
2. You are a beginner user stuck on something you completely don't know how to solve or you are experiencing unexpected behaviour: feel free to contact us at support@qbrobotics.com²⁰, you will receive the specific support you need as fast as we can handle it.
3. You are quite an expert user, everything has always worked fine, but now you have founded something strange and you don't know how to fix it: we will be glad if you open an Issue in the package of interest on [our Bitbucket](#)²¹.
4. You are definitely an expert user, you have found a bug in our code and you have also correct it: it will be amazing if you open a Pull Request in the package of interest on [our Bitbucket](#)²²; we will merge it as soon as possible.
5. You are comfortable with *qbrobotics* products but you are wondering whether is possible to add some additional software features: feel free to open respectively an Issue or a Pull Request in the package of interest on [our Bitbucket](#)²³, according to whether it is just an idea or you have already provided your solution.

In any case, thank you for using *qbrobotics*²⁴ solutions.

18. <https://bitbucket.org/qbrobotics/qbhand-ros/src>

19. <https://answers.ros.org/questions/>

20. <mailto:support@qbrobotics.com>

21. <https://bitbucket.org/qbrobotics/qbhand-ros/src>

22. <https://bitbucket.org/qbrobotics/qbhand-ros/src>

23. <https://bitbucket.org/qbrobotics/qbhand-ros/src>

24. <https://www.qbrobotics.com>

6.3 C++ API library

6.3.1 qbdevice C++ API v7

qbdevice C++ API v7 are available on qrobotics [Bitbucket public repository](https://bitbucket.org/qrobotics/qbdevice-api-7.x.x.git)²⁵. The [CMake](https://cmake.org/)²⁶ based project contains two *submodules*:

- **serial**: used to handle the communication with qrobotics devices.
- **qrobotics-driver**: used to control and set/get parameters from qrobotics devices.

Cloning and compiling

In order to clone the project, it is necessary to run the following command:

```
git clone --recurse-submodules https://bitbucket.org/qrobotics/qbdevice-api-7.x.x.git
```

Once the project is cloned, it is possible to compile qrobotics libraries by executing the following commands in *qbdevice-api-7.x.x* folder:

```
mkdir build
cd build
cmake ..
make
```

When the compilation is completed, the library binaries are within the folder *build/qrobotics-driver* and *build/serial*. They can be now used in your own application!

CMake project setup

In order to set up a CMake project, using the compiled files, that implements your own application, we suggest to create a new project that has a structure similar to the following:

```
├── include
│   ├── qrobotics_research_api
│   └── serial
├── libs
├── src
└── CMakeLists.txt
```

- the folder *include/qrobotics_research_api* must contain the following header files:

25. <https://bitbucket.org/qrobotics/qbdevice-api-7.x.x/src/production/>

26. <https://cmake.org/>

- **qbrobotics_research_api.h** - present in the folder *qbdevice-api-7.x.x/qbrobotics-driver/libs/research/include/qbrobotics_research_api* of qbdevice-api-7.x.x project.
- **qbrobotics_research_commands.h** - present in the folder *qbdevice-api-7.x.x/qbrobotics-driver/libs/research/include/qbrobotics_research_api* of qbdevice-api-7.x.x project.
- **qbsofthand_research_api.h** - present in the folder *qbdevice-api-7.x.x/qbrobotics-driver/libs/research/include/qbrobotics_research_api* of qbdevice-api-7.x.x project.
- the folder **include/serial** must contain the following header file:
 - **serial.h** - present in the folder *qbdevice-api-7.x.x/serial/include*
- the folder **libs** must contain the compiled libraries obtained by following the instructions in the previous paragraph.
- the folder **src** must contain the main.cpp that exploits qbrobotics API functionalities.
- The CMakeLists.txt has to configure the project to link against the compiled libraries.

Once the folders setup is finished you can compile your own application by executing the commands used to compile the libraries in the previous folder.



An example of how to structure the project can be found in the [Bitbucket repository API-test-SHR-v7²⁷](#). This project explains also how to handle the communication with *qb SoftHand Research* and how to control it.



The Research devices, manufactured by **qbrobotics**, are open source as the company's aim is to facilitate the work of our customers by allowing them to personalise the use of the device to their liking. Within the source code, however, there are functions that if used improperly can damage the device and invalidate its warranty status. What we ask is therefore that you only use the functions that you really need and make sure you fully understand how they work before you use them. Our Support Team is always at your disposal for any clarification you may need, you can contact them at support@qbrobotics.com²⁸ address.

6.3.2 qbdevice C++ API v6

Installation

To install all the requirements for your specific operating system, please refer to the [how-to section](#) (see [Page 40](#)).

To compile the C/C++ API library:

- open a terminal (for Linux and macOS users), or the command prompt (for Windows users);
- navigate to the `"src"` directory inside the downloaded package;
- execute the `"make"` command.

27. <https://bitbucket.org/qbrobotics/api-test-shr-v7/src/master/>

28. <mailto:support@qbrobotics.com>

Under `"lib_unix"` or `"lib_win"` directory you will find the newly created static library `"libqbmove_comm.a"` that can be linked in your own application.



NOTE:

Do not forget to include the `"qbmove_communications.h"` when linking the API library in your project.

Basic functions

In most cases these are all the necessary functions to integrate the qb SoftHand within your system.

- `int RS485ListPorts(char **serial_ports)` : retrieves all the serial ports connected to the system, returns their number, and fills the given argument with the connected serial port names;
- `void openRS485(comm_settings *file_descriptor, const char *serial_port)` : opens the serial communication by acquiring the serial port resource and sets the given file descriptor accordingly;
- `void closeRS485(comm_settings *file_descriptor)` : closes the serial communication and frees the serial resource;
- `void commActivate(comm_settings *file_descriptor, int device_id, char activate)` : activates (or deactivates) the motor on the qb SoftHand with the given id, respectively if `"activate"` is `"0x03"` (or `"0x00"`);
- `int commGetActivate(comm_settings *file_descriptor, int device_id, char *activate)` : fills `"activate"` with the activation status of the motor on the qb SoftHand with the given id, and returns 0 on success;
- `int commGetMeasurements(comm_settings *file_descriptor, int device_id, short int measurements[3])` : fills `"measurements"` with the position of the motor on the qb SoftHand with the given id, and returns 0 on success; only the first value is meaningful and it is expressed in motor ticks which lay in range `"0, 19000"`.
- `int commGetCurrents(comm_settings *file_descriptor, int device_id, short int currents[2])` : fills `"currents"` with the current of the motor on the qb SoftHand with the given id, and returns 0 on success; only the first value is meaningful and it is expressed in milliamperes.
- `void commSetInputs(comm_settings *file_descriptor, int device_id, short int inputs[2])` : sends a reference position command to the qb SoftHand with the given id; only the first value is meaningful and it is expressed in motor ticks which lay in range `"0, 19000"`.



CRITICAL:

These commands are meant to be used by expert users only.

Usage example

```
#include <iostream>
#include <string.h>
#include <unistd.h>
#include "qbmove_communications.h"

int main (int argc, char** argv) {
    char serial_ports[10][255];
    int serial_ports_count = RS485listPorts(ports);
    if (serial_ports_count <= 0) {
        std::cout << "ERROR: no serial port found" << std::endl;
        return -1;
    }
    for (int i=0; i<serial_ports_count; i++) {
        std::cout << "INFO : serial port " << serial_ports[i] << "found." << std::endl;
    }

    // ...

    comm_settings file_descriptor;
    openRS485(&file_descriptor, "/dev/ttyUSB0"); // use one of the values from
RS485listPorts
    if (file_descriptor.file_handle == INVALID_HANDLE_VALUE) {
        std::cout << "ERROR: fails while opening the serial resource (sets errno [" <<
strerror(errno) << "])." << std::endl;
        return -1;
    }

    // ...

    int device_id = 1;
    commActivate(&file_descriptor, device_id, TRUE); //activate motor
    usleep(10000);
    char status;
    int result = commGetActivate(&file_descriptor, device_id, &status);
    if (result != 0 || status != TRUE) {
        std::cout << "ERROR: fails while activating motor" << std::endl;
        return -1;
    }

    // ...

    short int currents[2]; // only the first value is meaningful
    short int positions[3]; // only the first value is meaningful

    if (commGetMeasurements(&file_descriptor, device_id, positions) < 0) {
        std::cout << "ERROR: fails while retrieving motor position" << std::endl;
        return -1;
    }
    std::cout << "INFO : motor position is " << positions[0] << std::endl;
```

```

if (commGetCurrents(&file_descriptor, device_id, currents) < 0) {
    std::cout << "ERROR: fails while retrieving motor current" << std::endl;
    return -1;
}
std::cout << "INFO : motor current is " << currents[0] << std::endl;

// ...

short int commands[2];
commands[0] = 10000; // nearly half closure (max is 19000, min is 0)
commands[1] = 0; // must be always 0 (only the first value is meaningful)
commSetInputs(&file_descriptor, device_id, commands);
usleep(1000000);

if (commGetMeasurements(&file_descriptor, device_id, positions) < 0) {
    std::cout << "ERROR: fails while retrieving motor position" << std::endl;
    return -1;
}
std::cout << "INFO : motor position is " << positions[0] << std::endl;

commands[0] = 0; // fully open (max is 19000, min is 0)
commands[1] = 0; // must be always 0 (only the first value is meaningful)
commSetInputs(&file_descriptor, device_id, commands);
usleep(1000000);

// ...

commActivate(&file_descriptor, device_id, FALSE); //deactivate motor
usleep(10000);
char status;
int result = commGetActivate(&file_descriptor, device_id, &status);
if (result != 0 || status != FALSE) {
    std::cout << "ERROR: fails while deactivating motor" << std::endl;
    return -1;
}

// ...

closeRS485(&file_descriptor);
return 0;
}

```

7 Maintenance and warranty

7.1 General

Products of the company qbrobotics s.r.l. are produced using the most modern production methods and are subject of strict quality inspections. All information regarding our Warranty Policy can be found at [this](https://qbrobotics.com/technical-support/)²⁹ site.

29. <https://qbrobotics.com/technical-support/>

8 FAQ and troubleshooting

For any issue not listed below, please contact our [support team](mailto:support@qbrobotics.com)³⁰.

8.1 PC drivers and requirements

8.1.1 OS requirements

The minimum hardware requirements ensure basic compatibility with the supported operating systems, while the recommended requirements provide a smoother and more stable experience, especially when running Windows 10.

A computer with a dual-core processor and at least 4 GB of RAM is sufficient for the necessary operations, but for better performance a quad-core processor with at least 8 GB of RAM is recommended. Using an SSD drive, although not mandatory, significantly improves operating system responsiveness and loading times.

In terms of connectivity, at least one USB port is required for FTDI drivers and a network connection (wired or wireless). A Full HD monitor is also recommended for improved readability of graphical interfaces.

All the Long-Term Support (LTS) releases of the following OS that have not reached the End-Of-Life (EOL) term are officially supported.

Component	Minimum	Recommended
CPU	Dual-core $\geq$ 2.0 GHz (Intel i3 4th gen or AMD eq.)	Quad-core $\geq$ 2.5 GHz (Intel i5/i7 6th gen or AMD eq.)
RAM	4 GB	8 GB or more
Storage	64 GB free (HDD)	128 GB+ SSD
Graphics	Integrated (Intel HD Graphics or eq.)	Recent integrated or entry-level GPU (e.g. NVIDIA)
Ports	1× USB 2.0/3.0	2+ USB (preferably 3.0)
Network	Integrated Ethernet or Wi-Fi	Ethernet + Wi-Fi
Monitor	1280×720	1920×1080 (Full HD)

30. <mailto:support@qbrobotics.com>

**WARNING:**

Using virtual machines (Windows or Linux) can be considered; however, this is not recommended. Its configuration could introduce issues such as increased latency and packet loss in network communications, potential incompatibilities with hardware drivers, and issues with system time synchronization..

Linux Ubuntu

Ubuntu 16.04, 18.04 and 20.04 are supported and recommended w.r.t. the other Linux distributions.

**NOTE:**

Note: different Linux distributions should also work fine, but they are not officially supported.

Microsoft Windows

As far as July 2020, Windows 10 version 1809+ and Windows 10 Enterprise 2015+ are supported and recommended.

8.1.2 Drivers installation

If you are using either Microsoft Windows or Apple macOS it is necessary to download and install the communication drivers from the [FTDI VCP Drivers](https://www.ftdichip.com/Drivers/VCP.htm)³¹ page.

Linux Ubuntu**LINUX FRIENDLY:**

You do not need additional drivers if you are running on any Linux distribution.

However, if you have never set it up, you probably need to add your linux user to the `dialout` group to grant right access to the serial port resources. To do so, just open a terminal and execute the following command:

```
sudo gpasswd -a <user_name> dialout
```

where you need to replace the `<user_name>` with your current linux username.

31. <https://www.ftdichip.com/Drivers/VCP.htm>

**NOTE:**

Note: don't forget to logout or reboot.

Microsoft Windows

Install the driver for your operating system version to allow the serial port access through something like ``COM7``.

**WARNING:**

Be aware that we support only serial port names between ``COM1`` and ``COM9``.

Each time you connect a new device to your PC (or even a device to a new PC), you should check whether it has been set in the proper port range or not.

To change the serial port number:

1. go under `"Control Panel > Hardware and Sound > Device Manager"`;
2. explore the `"Ports (COM & LPT)"` branch, right-click on the displayed ``COM`` port and select `"Properties"`;
3. from there, select the `"Port Settings"` tab and click on the `"Advanced"` button ;
4. lastly, choose from the drop-down menu a ``COM`` number between ``COM1`` and ``COM9`` and click on `"OK"`.

8.1.3 API installation requirements

Before using the qbAPI on your system, if not already installed it is good use to install a c++ compiler. Below here are listed all the necessary instructions for each operative system.

Linux Ubuntu**LINUX FRIENDLY:**

You should have both `"gcc/g++"` and `"make"` already installed if you are running on any Linux distribution.

Apple macOS

Download `"XCode"` from the App Store, this will install `"gcc/g++"` and `"make"` utilities.

Microsoft Windows

Download [MinGW](http://www.mingw.org)³² and install it. Open MinGW Installation Manager, from the left panel select basic setup, then from the right panel select mingw32-base and mingw32-gcc-g++, then click on **Installation** -> **Apply Changes**. This will install the *gcc/g++* compiler. To use it from the command line you need to provide to windows the binary path to the executable. Go to System Properties and click on Environment Variables. In the System Variable windows, look for **path**, select it and click **edit**. Go to the end of the Variable Value field, add a **;** separator and add the path to the binary folder for gcc (usually it is in **C:\MinGW\bin**).

Download the *make* utility from [here](http://gnuwin32.sourceforge.net/packages/make.htm)³³. Follow the installation instruction. In the end you will need to add the binary path to the Environment Variables. To do that follow the previous steps. (Usually the binary folder for the make utility is in **C:\Program Files (x86)\GnuWin32\bin**).



NOTE:

Note: if you have the CMD already opened when performing the installation, you probably will need to reopen a new one to be able to use the utilities.

8.2 Grasping with a custom finger position

8.2.1 Problem

The qb SoftHand adapts the finger position to envelop the object shape as much as possible — i.e. automatically.

However the mechanical synergy of the hand always goes from a fully open "*slap*" configuration to a fully closed "*punch*".

If you are wondering whether it is possible to command the hand to change this behavior or not, the answer is no.

8.2.2 Solution

As stated in the [description](#) (see [Page 13](#)) chapter of the manual "*The single-motor actuation makes the hand [...] simple-to-control (one single motor requires one single control signal to close and open the whole hand)*". Thus there is no way to control each finger independently. For such a purpose you would need several motors with the consequent increase in complexity of the control.

Anyway, since the qb SoftHand can grasp a huge variety of objects with just one synergy, we prefer to have a simple and powerful device w.r.t. a bit more versatility alongside with a complex control system behind.

Quite for sure you can find another way to perform the task you have in mind. Have a look at our suggested articles and try to exploit those tips.

32. <http://www.mingw.org>

33. <http://gnuwin32.sourceforge.net/packages/make.htm>

If you want some help with a specific task you will need to raise a "Feasibility Request" in our portal.

8.3 Simplified CAD models of the qb SoftHand

8.3.1 Problem

Where can I find CAD models or simplified meshes of the qb SoftHand Research?

8.3.2 Solution

We provides only simplified geometries of the qb SoftHand Research that you can find at our [ROS description repository](#)³⁴ (measures are expressed in millimeters).

As additional information, the phalanges are made of a polyamide 6 reinforced with 30% glass fiber (discontinuously placed), with the following main properties:

- density : 1.36 [g/cm³];
- tensile yield strength : 165 [MN/m²];
- tensile Young modulus : 8500 [MN/m²];
- percent elongation at break : 3%;
- resilience (Izod method) : 125 [J/m];
- melting point : 220 [°C];
- Poisson ratio : N/A (for general polyamide 6, i.e. without glass fiber, it is approximately 0.4).

8.4 The glove is deteriorated/broken

8.4.1 Problem

The glove is one of the component of the hand more prone to wear and tear.

The planned maintenance already makes up for this, but it may happen that the glove simply breaks during the normal usage.



WARNING:

Be aware that you must never grasp sharp objects, nor have them close to the robot during its operation, cf. [safety warnings](#) (see Page 4).

A damaged glove can affect repeatability and grasping performance.

34. https://bitbucket.org/qbrobotics/qbhand-ros/src/dd75e2f70bbb470be4a2360177dbde6a7afbd638/qb_hand_description/meshes/?at=production-kinetic

8.4.2 Solution

Unfortunately is not possible to ship you a whole new glove and let you proceed with its replacement.

Both the glove and the cover which secures it to the hand are crucial components of the hand and only assembling them inside our facilities guarantees the right setup and the advertised performance of grasping.



NOTE:

This is true also for the other elastic components inside the fingers.

In such a case you should raise a *"Service Request"* in our portal as soon as you notice the problem. We will do our best to ensure that this goes as quickly and as smoothly as possible.

8.5 The hand does not reopen correctly

8.5.1 Problem

The hand does not fully open up after setting the 0% position command: i.e. some of the fingers still remain a bit closed even if the motor position is correctly commanded to 0. Some of the fingers may move a bit slower than expected. Moreover, closure and reopening cycles present the same unexpected behavior at each open.



WARNING:

Be aware that this behavior can lead to hardware malfunctions if it is not solved quickly. If it happens too often, try to grasp the objects more gently (e.g. smaller closure values) or use another approach strategy which produce a more natural grasp.

8.5.2 Solution



USER-FRIENDLY:

This problem can be easily solved by the user in a couple of minutes!

The thumb phalanges are not properly placed: indeed it can happen that they don't always go back to the original position (this highly depends on how fingers envelop the object and how much each phalanx dislocate from its original place).

As you can see in the video I attached below, you can try to manually adjust the thumb configuration by pulling or stretching each phalanx.

If you make sure that thumb phalanges are aligned, you should have no more troubles.

If the thumb seems ok, just try to do the same to all the other fingers: pull and twist a bit each distal phalanx until it seems to be in the proper place (set the hand reference to 0%, i.e. fully open, when you do such maintenance).

**NOTE:**

It is a good practice to perform these manual adjustments once in a while to ensure that the hand works at its best.

Please, let us know whether this solve the problem or you need further assistance.

8.6 The USB flash drive is not working

8.6.1 Problem

It is not so common, but it may happen that any of the files provided through the USB flash drive is corrupted.

8.6.2 Solution

You can find all the USB flash drive files in our download section at the bottom of the [qb SoftHand page](#)³⁵.

**WARNING:**

If any of the files is corrupted we recommend to format the flash drive and download again all the files from our website.

35. <https://qbrobotics.com/products/qb-softhand-research>